

Hierarchical Clustering via Local Search

Lio Wu and Caspar Popova

Abstract

Hierarchical clustering is used in genomics to analyze genes by organizing data into a tree-like structure of clusters. In this project, we study an agglomerative local search algorithm for hierarchical clustering. Agglomerative methods do clustering by assigning each data point to its own cluster, then merging clusters. The local search method, rather than merging nodes, uses a tree re-arrangement operation known as interchange to swap two subtrees when the resulting tree is more profitable. In this project, we implement Jowhari's local search methodology of hierarchical clustering*. We compare the greedy variation of the local search algorithm to the average link, a common agglomerative clustering algorithm, by measure the revenue of the produced cluster. We demonstrate that our implementation does not match the revenue of average link, and analyze reasons why we fell short.

Introduction

Hierarchical clustering is used in genomics to analyze gene similarity by organizing data into a tree-like structure of clusters. In the tree, the root represents all data points, each node represents a cluster of similar data points, and the leaves are the data points. This is particularly useful for examining data at many different scales, e.g., individual cells to gene mutations. Hierarchical clustering can be viewed as an optimization problem, where we seek a tree with optimal revenue using some symmetric function $w : [n] \times [n] \rightarrow \mathbb{R}^+$ that specifies the pairwise similarity between n data points [1]. Given a clustering tree T over n leaves, where each leaf is numbered $\{1 \dots n\}$, we use the Moseley-Wang revenue function [2]. $w(i, j)$ is the similarity between 2 datapoints i and j . $T_{i,j}$ denotes the subtree that is rooted at the lowest common ancestor of i and j .

$$\text{rev}(T) = \sum_{i < j} w(i, j)(n - |T_{i,j}|)$$

The key challenge of hierarchical clustering is that computing the optimal tree (a tree having optimal revenue) is an NP-hard problem [3]. There are two major methods of hierarchical clustering, agglomerative and divisive, that seek to maximize the revenue. Agglomerative methods initially assign each data point to its own cluster and then merge clusters to build a tree from bottom up using some distance or similarity measure to decide which clusters are merged. Divisive methods start with one cluster containing all data points, and then split the clusters to build the hierarchy top-down [1].

Now, we discuss Jowhari's method of hierarchical clustering via local search [1]. The local search algorithm is more similar to agglomerative methods, where the next cluster "merged" is

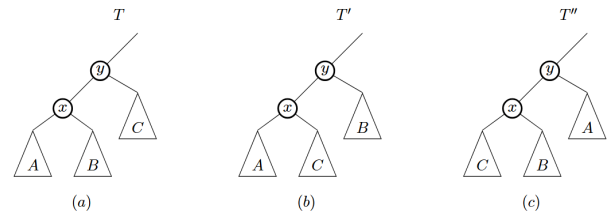


Figure 1 The interchange operation on trees (Figure from [1]).

determined by a search for the local change that produces the most revenue. In practice, though, Jowhari proceeds by initializing a random tree where the data points are the leaves and then either finding the most profitable interchange until the revenue can no longer be improved (the greedy local search method) or looking at random edges until there are no more profitable interchanges left (the random local search method). Concretely, given n data points, the local search algorithm initializes a binary unordered tree with n leaves and $2n - 1$ nodes, such that all data points are at a depth $\log(n)$ in the tree. The algorithm then relies on a tree re-arrangement operation, interchange, and compares the revenue of all possible interchanges in the current tree to decide which subtrees should be locally updated.

As shown in the Figure 1, the interchange operation applies to a subtree y , where y is the parent of x and x is a non-leaf and y is the parent of x . It yields one of the outcomes (b) or (c) in the same figure, where the subtree C , the second child of y , is swapped with either A or B . Importantly, this interchange assumes that the tree is unordered (i.e., it does not matter which child is the left and which is the right).

The theoretical bounds of this approach mirror the popular agglomerative approach, the average link algorithm [2]. In both cases, each algorithm is guaranteed to produce a tree with at least

*Our implementation is available at <https://github.com/iwu980/cmssc701-hcgorithms/tree/jowhari-v1>

1/3 of the maximum revenue, which is $(n - 2)\sum_{i < j} w(i, j)$. This is because in a maximal scenario, there would only be two leaves (i and j) in a subtree rooted at their shared lowest ancestor. However, a random tree's expected revenue is also 1/3 of the maximum revenue, which means that the lower bound here is not a particularly good one.[2] While the theoretical bounds are not particularly good, Jowhari's empirical results [1] suggest that the average revenue is much better than the guaranteed lower bound. However, Jowhari's chosen datasets do not compare to the scale of single-cell genomics data. Similarly, Jowhari's algorithm runs very well and converges relatively quickly on smaller datasets, with its $\mathcal{O}(n^2)$ preprocessing time and its $\mathcal{O}(n)$ interchanges, but its performance has not been demonstrated for larger datasets. This paper evaluates the relevance of Jowhari's method for large-scale genomics datasets through the metrics of convergence time and revenue.

In summary, in our project:

- We implement Jowhari's local search algorithm (Section 2) and discuss our implementation (Section 3).
- We compare our implementation to the average link algorithm, and analyze the differences between our results (Section 4).

Background: Local Search

In this section, we describe Jowhari's local search algorithm in detail. Jowhari's algorithm consists of three major steps: the preprocessing, finding the most profitable interchange, and updating the data structures.

Initialization

Given n data points, we instantiate an unordered binary tree with n leaves and $m = 2n - 1$ nodes of height t . Each node has a unique identifier u and T_u is the subtree rooted at u .

We instantiate a similarity matrix $W[i][j]$ that stores the similarity between each pair of nodes i, j in the tree. To populate $W[i][j]$, we partition the nodes (in $\mathcal{O}(n)$ time) into subsets H_i where H_i contains all the nodes that root a subtree of height i .

We use the Gaussian kernel, below, as the similarity function w with $\sigma = 1$. We begin by using w to compute $W[i][j]$ for every pair of leaves.

$$K(x, x') = e^{-\frac{\|x - x'\|^2}{2\sigma^2}}$$

Next, we must compute $W[i][j]$ for every pair of leaves i, j in $H_r \times H_{0 < j < r}$ for r from 1 to t . We use the following formula, where i_1, i_2 are the 2 children of i and j_1, j_2 are the children of j .

$$W[i][j] = W[i_1][j_1] + W[i_1][j_2] + W[i_2][j_1] + W[i_2][j_2]$$

If i happens to be a leaf, then i_1 and i_2 are substituted for itself; in other words, $W[i][j] = W[i][j_1] + W[i][j_2]$ when i is a leaf. The complexity of the initialization requires $\mathcal{O}(n^2)$ time and $\mathcal{O}(n^2)$ space.

Local Search and Interchange

Finding the most profitable interchange. To choose where to perform the interchange, we must first calculate the revenue interchange using the information in W for every edge. For every subtree T , as shown in Figure 1, we compute the revenue of the two possible interchanges that create trees T' and T'' as follows:

$$\text{REV}(T') = \text{REV}(T) + (|B|w(A, C) - |C|w(A, B))$$

$$\text{REV}(T'') = \text{REV}(T) + (|A|w(B, C) - |C|w(A, B))$$

In other words, because x remains rooted at this tree regardless of the interchange, we can calculate the increase in local revenue at the tree rooted at T . The subtrees A , B , and C have the same revenue and similarity scores within themselves, so by subtracting the revenue generated by that are no longer children of x while adding the revenue generated by new leaves, we can get the change in revenue for $\text{REV}(T)$. This is not necessarily the change in global revenue. Because all of the subtrees in these calculations remain children of the root, the similarity scores within subtrees and revenue within subtrees changes. However, by approaching locally optimal trees we can approximate the tree with maximum revenue. In iterating over every edge (y, x) , where x is a child of y with two children and y also has two children, the greedy local search algorithm will find the maximum value among every possible combination of subtrees between $(|A|w(B, C) - |C|w(A, B))$ and $(|B|w(A, C) - |C|w(A, B))$. This edge is subsequently interchanged. Because the steps done to evaluate each edge are constant time arithmetic, this gives us an $\mathcal{O}(n)$ search because the number of edges is directly proportional to the number of leaves.

Update operation on the matrices. Once we have selected the most profitable interchange and updated the tree itself, we must update W . The only subtree whose leaves have changed are x . Thus, it is sufficient to update entries that contain x . For each node i , where z is the old child of x and z' is the new child of x , we can just set $W[x][i]$ and $W[i][x]$ to be $W[x][i] + W[z'][i] - W[z][i]$. This update is done in $\mathcal{O}(n)$ time.

Theoretical complexity

Any HC tree of order n can be transformed into another HC tree of order n with $\mathcal{O}(n * \log(n))$ interchanges.

Jowhari's algorithm uses an array structure that takes $\mathcal{O}(n^2)$ space and $\mathcal{O}(n)$ time for each interchange.

Implementation

In this section, we discuss the concrete details of our implementation of Jowhari's local search algorithm.

We represent the binary tree with a collection of data structures where the nodes are 0-indexed, beginning from the leaves. Instead of using a traditional pointer structure, we use a map $n \rightarrow (n, n)$ from a node identifier u to its children identifiers u_1, u_2 to make the interchange process faster. We show an example of a computed HC dendrogram in in Figure 2. Additionally, we use the preprocessing structures identified by Jowhari: the similarity score matrix and the list of nodes by height. To make auxiliary calculations faster and simpler, we also store arrays with the data at each leaf, the height of each node, the parent of each node, and the number of leaves in the subtree rooted at that node. All of these structures require $\mathcal{O}(n)$ space, except for the similarity score matrix, which requires $\mathcal{O}(n^2)$ space. Similarly, the height partition, data at each leaf, parent of each node, and the number of subtrees rooted at each node can be computed in one $\mathcal{O}(n)$ pass and the height of each node can be calculated with an additional pass using the height partitions. The

similarity matrix dominates the time complexity, requiring $\mathcal{O}(n^2)$ time. From there, we repeatedly find the edge with the highest local interchange profit until we can no longer find such an edge, as described in Section 2 by iterating over all edges between a parent with two children and its child with two children. Then, we update the tree structure by shifting our map. Our height- and subtree-related auxiliary structures can be updated in one pass going up the tree in $\mathcal{O}(n)$ time, beginning from x and doing arithmetic with nodes on the path to the root. Additionally, we must update the similarity score matrix, which is proven possible in $\mathcal{O}(n)$ time in Section 2. However, because for large datasets, the algorithm often will not converge in a reasonable amount of time, we have the following additional halt conditions:

1. The global revenue is the same as it was 1,000 iterations ago,
2. We have reached 78,000 interchanges and the global revenue is less than it was 1,000 iterations ago,
3. The interchange involves swapping the same nodes as the last interchange, OR
4. We have reached 100,000 interchanges

This does influence the data shown in Section 4 by setting the halt points at even 1,000 intervals away from one another and explains the outliers at 78,000 interchanges. In order to calculate the global revenue, we developed the following algorithm:

1. Make clusters $C[n]$ of all leaves with parents that have a height of 1, where the height is defined as the longest path from a given node to a leaf. Each parent n has its own set of leaves.
2. Select a cluster $C[n]$. If the sibling of n , call this node s , is not a leaf and not in the list of cluster roots, pick a different cluster.
3. If the sibling is a leaf, then make $C[s]$ a list of size 1 containing s .
4. We know that $|C[n]| + |C[s]|$ is the number of leaves in the tree rooted at the lowest common ancestor of any pair of leaves i, j where $i \in C[n]$ and $j \in C[s]$ because the parent p of n and s is the lowest common ancestor. Thus, we can add $(L - |C[n]| - |C[s]|)w(i, j)$ to our revenue, where L is the total number of leaves and $w(i, j)$ is the similarity function between i and j .
 - We are guaranteed to only see each pair once because no leaf can appear in two disjoint subtrees.
5. Now, remove $C[n]$ and $C[s]$ from our list of clusters and add $C[p] = C[n] \cup C[s]$ to our list of clusters.
6. Repeat this process until we reach the root.
 - A small amount of time can be saved before merging the children of the root because they will not contribute to the revenue. The total number of leaves L is equal to the number of leaves rooted at the root, so we get $n - |T_{i,j}| = 0$.

The time complexity is the number of time it takes to iterate through every pair of nodes, so $\mathcal{O}(n^2)$.

Evaluation

In this section we present empirical results related to the local search algorithm. We use 4 datasets from the Tabula Sapiens corpus [4], listed in Table 1 with the number of cells in each dataset. We utilize the UMAP embeddings of the data. UMAP [5] is a dimension reduction technique that reduces the original data, which can contain

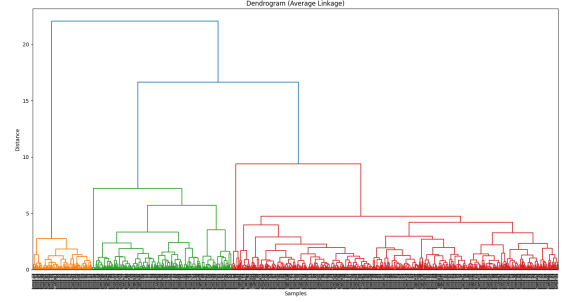


Figure 2 Generated HC tree for sample from Gland Immune Atlas dataset.

Table 1 Specification of data used in the experiment.

Dataset	Size
Dendritic cells	15063
Gland Immune Atlas	6362
PBMC Other	93541
SMBO-114	2410

thousands of genes for each cell, to a smaller number of embeddings. Here, we use dimension 2 such that each cell is represented as 2 numbers (x, y) . We initially chose these 4 datasets as they vary in scale, however we used samples of 1000 cells in our testing due to limitations of local compute. Samples were taken consistently across runs.

We compare our implementation of the local search algorithm (LS) using greedy search against an existing implementation of the average link algorithm (AL) ¹. For each dataset, we record the revenue, where the local search revenue is averaged over 5 independent runs: on each run, we initialize the tree randomly by shuffling the input vectors, which generates a different complete binary tree where leaves have different initial siblings. To normalize revenue against data size, we divide it by $(n - 2) \sum_{i < j} w(i, j)$.

As shown in Table 2, we observe that average link tends to do better on datasets of this scale, while average link struggles. Additionally, these means are considerably worse than the average revenue Jowhari was able to achieve on a smaller dataset.[1] This is likely because as the dataset grows, the effect of the gains from local interchanges on the total revenue decreases and can even cause the global revenue to decrease in some scenarios. Additionally, in many scenarios, the algorithm had to be halted either because it was cycling through a few interchanges and global revenues or making minimal progress. This is particularly visible when we examine the number of interchanges and the number of halts. For larger datasets, like the dendritic cells dataset and the SMBO-114, not only is the average number of interchanges higher, but the mean halting point is also much higher, as we can see in Table 2 and Table 3. As the datasets increase in size, the algorithm is less likely to naturally converge. Where Jowhari's greedy local search converged with an average of 2011 interchanges over all their datasets [1], the median number of interchanges for our data was 6500, as shown in Figure 3.

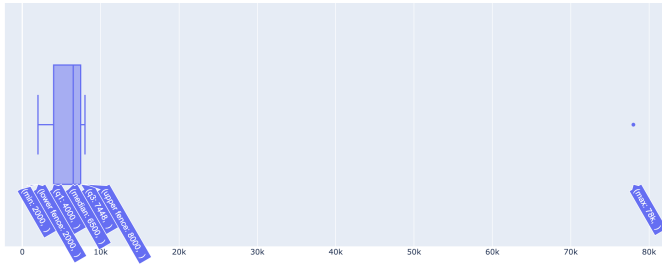
¹ <https://github.com/MajidJafari/Dasgupta-s-Cost-Function-And-Moseley-Wang-Revenue-Function/tree/main>

Table 2 Comparison of practical revenue from average link (AL) and greedy local search (LS) algorithms.

Dataset	AL	LS (Avg)	LS (# Interchanges)
Dendritic cells	0.84	0.36	20941
Gland Immune Atlas	0.91	0.37	4920
PBMC Other	0.65	0.35	6000
SMBO-114	0.90	0.35	5109

Table 3 Halts by dataset.

Dataset	Number of LS Halts	Mean Halting Point
Dendritic cells	4	24250
Gland Immune Atlas	3	3333
PBMC Other	4	6250
SMBO-114	4	4500

**Figure 3** Boxplot of interchanges

Related Work

As discussed in Section 2, our work is to make an open-source implementation of Jowhari’s local search method for hierarchical clustering [1].

We direct the reader to the survey paper by Mosely and Wang [6] for an overview of different hierarchical clustering algorithms and their theoretical guarantees. In particular, their work includes another local search algorithm, where local search is used in a divisive hierarchical clustering where the strategy determines which clusters are split. Unlike this algorithm, Jowhari employs local search in an agglomerative bottom-up method.

The interchange tree operation has also been used by Kobren et al. and Monath et al [7, 8] in incremental algorithms for hierarchical clustering, where the full set of data points is not available ahead of time (new data points are added and the clustering must be updated incrementally).

Average link is another agglomerative method that uses the average distance between all points from two clusters to decide which clusters to merge next [2]. In other words, for each pair of clusters, the average link algorithm computes the distance between the two clusters as the average distance between all the clusters’ points. Then, the two clusters with the smallest average distance are merged.

Future work

Matching Jowhari’s results. We tested our implementation of Jowhari’s algorithm on a different set of datasets. If our implementation also produces HS trees of lower revenues on the same

datasets, then we must analyze which heuristics we used cause our method to be suboptimal.

Convergence time. As mentioned by Jowhari, the convergence time of the local search algorithm compared to other hierarchical clustering methods like average link is unknown. There may be some theoretical upper bound on the number of interchanges necessary to reach a locally optimal tree. Empirical testing, which depends on the initialization of the tree, is also crucial to understanding the practical comparison between methods.

Combining HC methods. The practical revenue of local search is dependent on the input tree. It is possible that doing some pre-processing, or combining local search with another method that initially builds an HC tree, can yield a more optimal than using local search alone on random trees. It may also be productive to combine the local search method with agglomerative methods to optimize clusters of nodes before merging them into a tree.

Conclusion

In this project, we study Jowhari’s agglomerative local search algorithm for hierarchical clustering. This method searches for local swaps of subtrees in the tree, known as interchanges, that increase the revenue of the tree. We implement Jowhari’s local search method and compare it to the average link clustering algorithm. We demonstrate that our implementation produces trees with lower revenue than average link, and analyze reasons why this occurred.

References

1. Hossein Jowhari. Hierarchical clustering via local search, 2024. URL <https://arxiv.org/abs/2405.15983>.
2. Benjamin Moseley and Joshua R. Wang. Approximation bounds for hierarchical clustering: average linkage, bisecting k-means, and local search. *J. Mach. Learn. Res.*, 24(1), January 2023. ISSN 1532-4435.
3. Sanjoy Dasgupta. A cost function for similarity-based hierarchical clustering. In *Proceedings of the Forty-Eighth Annual ACM Symposium on Theory of Computing, STOC ’16*, page 118–127, New York, NY, USA, 2016. Association for Computing Machinery. ISBN 9781450341325. doi: 10.1145/2897518.2897527. URL <https://doi.org/10.1145/2897518.2897527>.
4. Stephen R Quake and The Tabula Sapiens Consortium. Tabula sapiens reveals transcription factor expression, senescence effects, and sex-specific features in cell types from 28 human organs and tissues. *bioRxiv*, 2024. doi: 10.1101/2024.12.03.626516. URL <https://www.biorxiv.org/content/early/2024/12/04/2024.12.03.626516>.
5. Leland McInnes and John Healy. Umap: Uniform manifold approximation and projection for dimension reduction. *ArXiv*, abs/1802.03426, 2018. URL <https://api.semanticscholar.org/CorpusID:3641284>.
6. Fionn Murtagh and Pedro Contreras. Algorithms for hierarchical clustering: an overview. *WIREs Data Mining and Knowledge Discovery*, 2(1):86–97, 2012. doi: <https://doi.org/10.1002/widm.53>. URL <https://wires.onlinelibrary.wiley.com/doi/abs/10.1002/widm.53>.
7. Ari Kobren, Nicholas Monath, Akshay Krishnamurthy, and Andrew McCallum. A hierarchical algorithm for extreme clustering. In *Proceedings of the 23rd ACM SIGKDD*

International Conference on Knowledge Discovery and Data Mining, KDD '17, page 255–264, New York, NY, USA, 2017. Association for Computing Machinery. ISBN 9781450348874. doi: 10.1145/3097983.3098079. URL <https://doi.org/10.1145/3097983.3098079>.

8. Nicholas Monath, Ari Kobren, Akshay Krishnamurthy, Michael R. Glass, and Andrew McCallum. Scalable hierarchical clustering with tree grafting. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, KDD '19, page 1438–1448, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450362016. doi: 10.1145/3292500.3330929. URL <https://doi.org/10.1145/3292500.3330929>.

Datasets

Permanent download links from Tabula Sapiens:

<https://datasets.cellxgene.cziscience.com/c6ea3545-9200-4497-8591-08f687626182.h5ad>
<https://datasets.cellxgene.cziscience.com/e9078993-a985-4159-b770-a0baf954d7f6.h5ad>
<https://datasets.cellxgene.cziscience.com/982c7fb5-a191-4125-8ff5-edea84790468.h5ad>
<https://datasets.cellxgene.cziscience.com/3b2878a8-4798-4ae0-b284-e77ed7c1b6a5.h5ad>